

Algorithm 1025: PARyOpt: A Software for Parallel Asynchronous Remote Bayesian Optimization

BALAJI SESA SARATH POKURI, Department of Mechanical Engineering, Iowa State University
ALEC LOFQUIST, Department of Computer Engineering, Iowa State University
CHAD RISK, Department of Chemistry, University of Kentucky
BASKAR GANAPATHYSUBRAMANIAN, Department of Mechanical Engineering,
Iowa State University

PARyOpt¹ is a Python based implementation of the Bayesian optimization routine designed for remote and asynchronous function evaluations. Bayesian optimization is especially attractive for computational optimization due to its *low cost function footprint* as well as the ability to account for uncertainties in data. A key challenge to efficiently deploy any optimization strategy on distributed computing systems is the synchronization step, where data from multiple function calls is assimilated to identify the next campaign of function calls. Bayesian optimization provides an elegant approach to overcome this issue via asynchronous updates. We formulate, develop and implement a parallel, asynchronous variant of Bayesian optimization. The framework is robust and resilient to external failures. We show how such asynchronous evaluations help reduce the total optimization wall clock time for a suite of test problems. Additionally, we show how the software design of the framework allows easy extension to response surface reconstruction (Kriging), providing a high performance software for autonomous exploration. The software is available on PyPI, with examples and documentation.

CCS Concepts: • **Mathematics of computing** → *Bayesian nonparametric models*; • **Applied computing** → **Mathematics and statistics**; • **Theory of computation** → **Non-parametric optimization**; • **Computing methodologies** → Optimization algorithms;

Additional Key Words and Phrases: Bayesian optimization, parallel optimization, remote evaluation, python software

ACM Reference format:

Balaji Sesa Sarath Pokuri, Alec Lofquist, Chad Risko, and Baskar Ganapathysubramanian. 2022. Algorithm 1025: PARyOpt: A Software for Parallel Asynchronous Remote Bayesian Optimization. *ACM Trans. Math. Softw.* 48, 2, Article 24 (July 2022), 15 pages.
<https://doi.org/10.1145/3529517>

¹PARyOpt is pronounced as /per'japt/. It means competent and optimal in Hindi.

Authors' addresses: B. S. S. Pokuri, Department of Mechanical Engineering, Iowa State University, 2029 Black Engineering, 2529 Union Drive, Ames, Iowa, USA - 50011; A. Lofquist, Department of Computer Engineering, Iowa State University, 2029 Black Engineering, 2529 Union Drive, Ames, Iowa, USA - 50014; C. Risko, Department of Chemistry, 276 CAER Laboratory 2, 150 Chemistry-Physics Building, Lexington, Kentucky, USA - 40506; B. Ganapathysubramanian (corresponding author), Department of Mechanical Engineering, Iowa State University, 2080 Black Engineering, 2529 Union Drive, Ames, Iowa, USA - 50011.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2022 Copyright held by the owner/author(s). Publication rights licensed to ACM.

0098-3500/2022/07-ART24 \$15.00

<https://doi.org/10.1145/3529517>

1 INTRODUCTION

Simulation-based design is a resource efficient approach for solving inverse and/or design problems in science and engineering. The goal of inverse/design problems is to identify conditions – these can be initial conditions, boundary conditions or property/coefficient distribution – that result in a desired behavior of the engineered system. Examples of these are ubiquitous. A good example is in the identification of tailored processing conditions that result in electronic devices with high performance metrics. In most electronics manufacturing, it has been shown that varying processing conditions can critically impact device properties and the identification of optimal processing conditions is a key problem from the financial and sustainability standpoint [15, 23, 24]. Another example is in the identification/design of useful molecular architectures that exhibit a suite of desired physicochemical properties (absorption, miscibility, solubility, toxicity, among others). This has substantial implications in the pharmaceutical and the chemical industries. In all such cases of simulation-based engineering, considerable effort has been expended to construct excellent “forward” models of the engineering problem, i.e., models that map the set of input conditions, boundary conditions and property distributions to the output quantity of interest ($\mathcal{F}$: input $\rightarrow$ property). The design problem, however, calls for knowledge of the **inverse mapping** ($\mathcal{F}^{-1}$: property $\rightarrow$ input) that maps a desired value of the output to a set of inputs.

Explicit construction of the inverse mapping is unfeasible in most applications with complex forward models. The design problem of identification is usually posed as an optimization problem. That is, the forward model, $\mathcal{F}$, is solved multiple times within an optimization framework to identify those input conditions that minimize a cost function, with the (argument of the) minima representing the desired input values. In complex engineering applications, this cost function is often very tedious to calculate. It can involve simulations which are computationally expensive. In such situations, these ‘forward’ simulations are performed on large **high performance computing (HPC)** resources. This federated approach to simulations is quite common, and gives several benefits such as parallelization and cost effectiveness. However, such large shared compute resources often work with a job scheduler that balances the load across several processes, users and projects, leading to uncertain times of completion (queueing time + initialization time + compute time) for each simulation. Added to that is the logistical effort of managing data between a local machine and an HPC client. Therefore, in such applications, one can expect the cost function to: (a) be solved on an HPC cluster, (b) have long, uncertain compute times, (c) require some form of data transfer across computers, and (d) fail with a small (but finite) probability due to hardware/network failures. In recent years, there have been various approaches proposed to solve optimization problems under such constraints.

Bayesian optimization (BO) is one such approach, especially for expensive cost function optimization. BO works by adaptively sampling the input parameters to construct a surrogate of the forward model through basis expansion. By efficiently constructing a surrogate, it achieves two purposes: one, to interpolate the surrogate through all existing data points and two, to give confidence estimates on the constructed surrogate. Furthermore, this information provides a rigorous way to determine the next set of evaluations/experiments (the adaptive sampling). This form of surrogate construction helps to directly reduce the total number of function evaluations and iterations, helping to efficiently identify the optima. Additionally, the Gaussian (mean, variance) description used for construction of the surrogate, enables a diversity of searches ranging from pure exploration to pure exploitation. Exploration helps to build confidence in the constructed surrogate and exploitation takes advantage of this confidence to find optima. A mixture of exploration and exploitation avoids the optimizer to be stuck at a local optimum. Hence, efficient BO usage involves strategic, possibly simultaneous, exploration and exploitation searches. This parallelized cost function evaluation can often increase the total time of optimization, often due to delays in resource

availability and management. BO is superior to other algorithms in this aspect, since the update of the surrogate is Gaussian and hence the order of data assimilation does not affect the posterior surrogate. Additionally, asynchronous data assimilation [6] can also be incorporated into this algorithm, with unusually long function evaluations (could be due to difficult convergence, hardware failures and several other logistic reasons) being assimilated in later iterations. Another common challenge with integration with an HPC system is to harmonize the several runs across simulation nodes. One may also be interested to post-process simulation results to calculate the actual cost function value. That will require a complete setup to manage data across several systems, quite often using a secure shell protocol.

Utilizing these advantages and addressing the concerns, we present a parallelized implementation of Bayesian optimization that can: (a) perform asynchronous updates per iteration, (b) comes with a **secure shell login (SSH)** module for HPC integration, and (c) has fault tolerant restart capability.

Using PARyOpt, the user can thus perform optimization on a local machine, with the flexibility to perform simulations either on the same local machine or a remote machine with/without a job scheduler. To tackle delays in simulation time, we provide an asynchronous evaluator class that can manage and keep track of status of simulations and adaptively assimilate them with completion. To tackle hardware/software failures, we provide functionality to restart optimization in case of head-node failure and also functionality to re-evaluate cost function in case of simulation failures. In the presented test cases, the asynchronous remote evaluator has improved the total time of optimization by up to 50%. Additionally, the software can also be used for general surface reconstruction, commonly known as Kriging. The rest of the paper is structured as follows – in Section 2, we discuss the basic math behind surrogate construction, Bayesian update and acquisition function, and potential avenues of parallelization. Section 3 discusses current implementation of the algorithm, and development and usage of the asynchronous evaluator class. In Section 4, several standard optimization problems as well as kriging are considered along with a thorough analysis of the novel asynchronous evaluator class. Finally in Section 5, code availability, reproducibility and future directions of this software are discussed.

2 BAYESIAN UPDATE – GAUSSIAN PROCESSES

In this section, we present a brief discussion of the algorithm of Bayesian optimization. The core algorithm is well studied and not the topic of this work. We refer the reader to [2, 19] for a detailed explanation of the algorithm, its requirements and limitations.

We consider a general minimization problem:

$$\min_{\mathbf{x}} y(\mathbf{x}) \quad (1)$$

Bayesian optimization proceeds through construction of a surrogate cost function $\tilde{y}(\mathbf{x})$. This surrogate is represented via a basis function expansion, around each evaluated point $(\mathbf{x}_i, i = 1, \dots, N)$. This ensures that the surrogate passes through (interpolates) the evaluated points. In the case of evaluations with noisy data, the surrogate shall pass within *one* standard deviation from the mean at the evaluated points. Analytically, the surrogate $\tilde{y}(\mathbf{x})$ after N function evaluations is represented as

$$\tilde{y}(\mathbf{x}) = \sum_{i=1,2,\dots,N} w_i k(\mathbf{x}_i, \mathbf{x}) \quad (2)$$

Typically, $k(\mathbf{x}_i, \mathbf{x})$ is a kernel function (Section 2.1), i.e., it takes in two arguments, $\mathbf{x}$, $\mathbf{x}_i$, and returns a scalar representative of correlation of the function $y(\mathbf{x})$ between the points $\mathbf{x}$ and $\mathbf{x}_i$. The weights w_i are calculated by solving the system of N linear equations in w_i . In matrix notation, this is represented using a covariance matrix ($\mathbf{K}$):

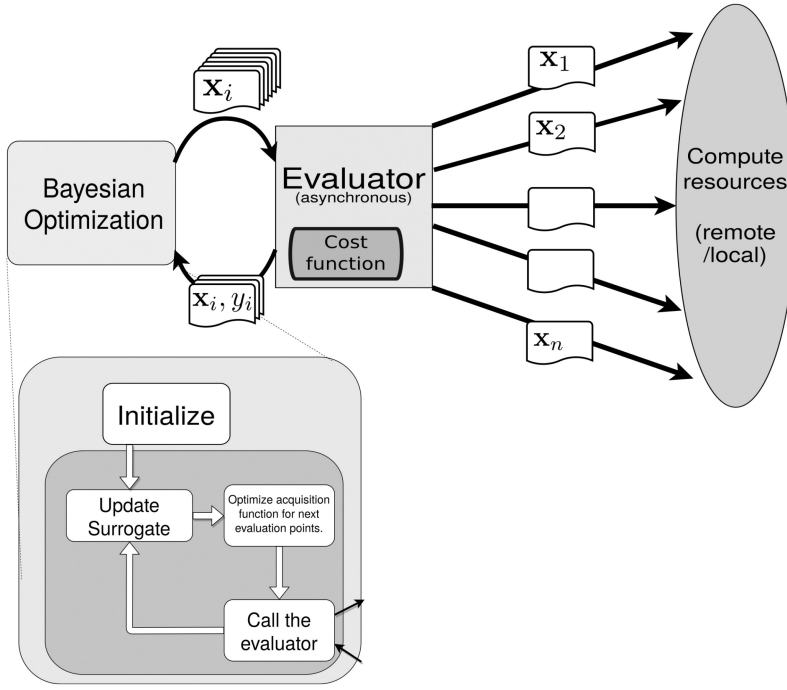


Fig. 1. Modular design of Bayesian optimization. Note how the evaluator can be designed with custom capabilities, such as asynchronicity and remote evaluation. The next figure will explain the implementation of an asynchronous evaluator.

$$\mathbf{K} \bar{\mathbf{w}} = \mathbf{y} \quad (3)$$

$$\mathbf{K}_{i,j} = k(\mathbf{x}_i, \mathbf{x}_j), \quad i, j \in [1, N] \quad (4)$$

$$y_i = y(\mathbf{x}_i), \quad i \in [1, N] \quad (5)$$

$$\bar{\mathbf{w}} = \{\mathbf{w}_i\}, \quad i \in [1, N] \quad (6)$$

Hence, the weights are calculated through the inversion $\bar{\mathbf{w}} = \mathbf{K}^{-1} \mathbf{y}$. Note that the covariance matrix $\mathbf{K}$ is a Gram matrix of a positive definite kernel function, making it symmetric and positive semi-definite (see Section 2.1). Furthermore, since with every iteration only a finite number of rows are added to the covariance matrix, efficient inversion is possible through incremental Cholesky decomposition [16]. The mean and variance of the surrogate are then calculated as:

$$\mu(\mathbf{x}_{N+1}) = \mathbf{k}^T \mathbf{K}^{-1} \mathbf{y}_{1:N} \quad (7)$$

$$\sigma^2(\mathbf{x}_{N+1}) = k(\mathbf{x}_{N+1}, \mathbf{x}_{N+1}) - \mathbf{k}^T \mathbf{K}^{-1} \mathbf{k} \quad (8)$$

where

$$\mathbf{k} = \mathbf{k}(\mathbf{x}_{N+1}) = [k(\mathbf{x}_1, \mathbf{x}_{N+1}) \ k(\mathbf{x}_2, \mathbf{x}_{N+1}) \ \dots k(\mathbf{x}_N, \mathbf{x}_{N+1})] \quad (9)$$

At each iteration, the surrogate is updated with new data from the cost function. The locations where the next evaluation is done is determined through optimization of an *acquisition function* (Section 2.2). An acquisition function is a means to estimate the new information content at a location. It uses the mean and variance calculated in the above steps. In total, the algorithm is summarized in the following figure (Figure 1).

2.1 Kernel Function

The kernel function, $f_i(\mathbf{x}) = k(\mathbf{x}, \mathbf{x}_i)$, is the most important component of Bayesian optimization. It embeds information about continuity, periodicity and correlation lengths of the underlying cost function, y . In order to be used as a kernel function in Gaussian processes the function should be a positive semi-definite function, i.e.,

$$\int k(\mathbf{x}, \mathbf{x}') g(\mathbf{x}) g(\mathbf{x}') d\mu(\mathbf{x}) d\mu(\mathbf{x}') \geq 0 \quad (10)$$

for all $g \in L(\chi, \mu)$, μ is any measure. An exhaustive description of the requirements of covariance functions can be found in [4]. For any distance metric, $r = \|\mathbf{x} - \mathbf{x}'\|$, some examples of standard stationary covariance functions include:

- (1) Squared Exponential covariance function

$$k_{SE}(r) = \exp\left(-\frac{r^2}{l^2}\right) \quad (11)$$

- (2) Matern class of covariance functions

$$k_{Matern}(r) = \frac{2^{1-\nu}}{\Gamma(\nu)} \left(\frac{\sqrt{2\nu}r}{l}\right)^\nu K_\nu\left(\frac{\sqrt{2\nu}r}{l}\right) \quad (12)$$

where K_ν is the modified Bessel function, ν, l are positive constants

- (3) Exponential covariance function

$$k_E(r) = \exp\left(-\frac{r}{l}\right) \quad (13)$$

- (4) γ -exponential function

$$k_{gE}(r) = \exp\left(-\left(\frac{r}{l}\right)^\gamma\right) \quad (14)$$

- (5) Rational quadratic covariance function

$$k_{RQ}(r) = \left(1 + \frac{r^2}{2\alpha l^2}\right)^{-\alpha} \quad (15)$$

- (6) Piecewise polynomials with compact support: Several algorithms are discussed in [22], one such example with compact support in D dimensions is given by

$$k_{ppD,0} = \left(1 - \frac{r}{l}\right)_+^j \quad (16)$$

where $j \geq \lfloor \frac{D}{2} \rfloor + 1$.

While several classes of kernel functions can be used, in our implementation, we focus on stationary functions (radial basis functions). It contains some of the most commonly used kernel functions like squared exponential, Matern 3/2 and Matern 5/2. It is implemented as a separate kernel function class, so that users can derive from this class and supply their own custom kernel functions. This class also contains the functionality to set the length scale parameters. This functionality is used to estimate and tune the current length scales for the construction of a better surrogate with a given class of kernel functions.

2.1.1 Maximum Likelihood Estimate. The **Maximum Likelihood Estimate (MLE)** for a Gaussian process gives information about the goodness of the surrogate based on the current available data. This provides a rigorous means of estimating the length scale in the kernel function. Generally known as hyper-parameter estimation, this is done by minimizing the MLE of the surrogate [13]. The MLE is defined as:

$$\begin{aligned} MLE &= \log(y^T \mathbf{K}^{-1} y) + \frac{1}{N} \log(\det(\mathbf{K})) \\ &= \log(y^T \mathbf{K}^{-1} y) + \frac{1}{N} \sum_{i=1}^N \log(\lambda_i(\mathbf{K})) \end{aligned} \quad (17)$$

The right hand side of Equation (17) can be broken into two terms: the first term quantifies how well the model fits the data, which is simply a Mahalanobis distance [11] between the model predictions and the data; and the second term quantifies the model complexity – smoother covariance matrices will have smaller determinants and therefore lower complexity penalties. Parameters of kernel function that minimize this metric are the best set for the constructed surrogate. It should be noted here that premature kernel parameter optimization can often lead to highly skewed estimates and MLE estimate should be performed only after a sufficient number of data points are evaluated [1, 3]. A detailed discussion of hyper-parameter estimation in Bayesian optimization can be found in [7, 19, 21].

2.2 Acquisition Function

The acquisition function informs the selection of the next point for evaluation of the cost function, y . It is also sometimes called the ‘in-fill’ criterion. The acquisition function is expected to locate the most useful point for evaluation of the cost function. This is particularly necessary when dealing with very expensive cost functions. The acquisition function operates on the mean and variance of the surrogate, $\tilde{y}$, and identifies the ‘best’ point for next evaluation. Typically, these functions are defined such that low acquisition values corresponds to (a) low values of cost function (*exploitation*), (b) high value of uncertainty (*exploration*), and (c) a balance between exploration and exploitation. Minimizing the acquisition function is used as a guide to select the next point of evaluation. Some of the commonly used acquisition functions that are implemented in the software are given below:

- (1) Lower Confidence Bound: Based on the mean and variance of the surrogate, the lower confidence bound of the surrogate is defined as

$$LCB(\mathbf{x}) = \mu(\mathbf{x}) - \kappa \sigma(\mathbf{x}) \quad (18)$$

κ is a manually tunable parameter that signifies exploitative search for smaller values and explorative search for large values.

- (2) Probability of Improvement: This estimates the probability of improvement of the surrogate at a given location. It is given by:

$$PI(\mathbf{x}) = \begin{cases} \phi\left(\frac{f_{min} - \mu(\mathbf{x})}{\sigma(\mathbf{x})}\right), & \text{if } \sigma(\mathbf{x}) > 0 \\ 0, & \text{if } \sigma(\mathbf{x}) = 0 \end{cases} \quad (19)$$

where

$$I(\mathbf{x}) = \min \left\{ \frac{f_{min} - \mu(\mathbf{x})}{\sigma(\mathbf{x})}, 0 \right\}$$

is the effective improvement at a given location and $\phi(\mathbf{x})$ is the normal probability distribution function.

- (3) Expected Improvement: This metric estimates the expectation of improvement of the surrogate at a location, given by:

$$EI(\mathbf{x}) = \begin{cases} \mu(\mathbf{x}) + (f_{min} - \mu(\mathbf{x}))\Phi\left(\frac{f_{min} - \mu(\mathbf{x})}{\sigma(\mathbf{x})}\right) + \sigma(\mathbf{x})\phi\left(\frac{f_{min} - \mu(\mathbf{x})}{\sigma(\mathbf{x})}\right), & \text{if } \sigma(\mathbf{x}) > 0 \\ 0, & \text{if } \sigma(\mathbf{x}) = 0 \end{cases} \quad (20)$$

Similar to lower confidence bound, expected improvement can also be parametrized for directing the optimization to exploration vs exploitation. This modified EI is called **generalized EI (g-EI)** and is very elaborately explained in [18].

2.3 Parallelization

There are several strategies for parallelization of this algorithm [17, 20]. It could be parallelization of (a) linear algebra during covariance matrix construction; (b) acquisition function optimization for multiple optima/exploration-exploitation; (c) cost function *execution*; and (d) cost function evaluation through multiple worker threads. Of these, the first two strategies give little computational advantage as the time spent in those stages is much smaller than expensive cost functions. Therefore, we take the route of parallelizing cost function execution and evaluation. This is especially critical and useful for HPC based evaluations, which have the ability to run simultaneous multi-processor jobs. Also, parallelizing function evaluation creates a natural setup for asynchronous data assimilation. Since the update is Bayesian, the order in which the data is added to the original prior is insignificant. The choice of the next function evaluation depends exclusively on the data and not the ordering of the data. This can also be seen in the construction of the covariance/correlation matrix, which is symmetric and the order of data only affects the order of rows and columns and has no effect on the characteristics of the matrix. This property of the algorithm enables asynchronicity to be built into it. The chosen ‘in-fill’ points for evaluation can be added later in the case of technical delays (typically long queue times or hardware failures on an HPC cluster) without affecting the progress of optimization. How much later and what the minimum number of evaluations per iteration form a part of the analysis of this algorithm which we present in the results section.

3 ASYNCHRONOUS FUNCTION EVALUATION

In order to abstract the cost function evaluation from the platform of evaluation, we develop a cost function evaluator class (Figure 2). The *evaluator* is a special function that takes two parameters: a list of “new” points to evaluate, and a list of “old”, pending points to include once evaluation has completed. The function evaluator returns three lists: a list of completed points (and their cost function value), a list of pending points (that are still being evaluated), and a list of points that failed to be evaluated. The union of the two lists of input points is always equal to the union of the three lists of output points.

The most straightforward implementation of a cost function evaluator is one that loops through all input points and evaluates the cost function at each point serially. We can extend this to evaluate points in parallel by spawning worker threads (or processes) that work through the list of input points in parallel until every point has been evaluated, then returning the results. In both cases, the returned pending list is always empty: all points either complete successfully or fail. We refer to both of these implementations as “synchronous” because the Bayesian optimization algorithm does not continue to the next iteration until all input points have been evaluated.

As cost function evaluation time may vary significantly, we introduce a tuneable parameter that controls the fraction of “new” points per iteration that *must complete* before the function evaluator returns. A value of 1.0 mimics synchronous/blocking behavior: all points must be evaluated before



Fig. 2. **Asynchronous evaluator:** The evaluator receives two lists of pending points and new points for evaluation. It returns three lists – evaluation finished, pending points, and failed evaluation. The fraction of evaluations finished can be controlled by the fraction of asynchronicity (blocking-fraction).

the function evaluator returns, and the pending list will always be empty. A value of 0.6 waits for at least 60% of points from the current iteration to be evaluated before continuing to the next iteration, returning the remaining points in the pending list. A value of 0.0 will begin evaluating the input points, then return immediately with all input points in the pending list. Only the “new” points (the first argument of the function evaluator) are considered part of this fraction to prevent a buildup of difficult-to-evaluate stale points holding up an iteration.²

Future calls to the function evaluator pass the previously-returned pending points list as the second argument (“old” points). Once the evaluator finishes evaluating the list of “new” points, it checks to see if any of the pending “old” points have completed (or failed). If they have, they are added to the appropriate list. If they have not, they are added to the pending list. The evaluator never waits for “old” points.

3.1 Implementation Details

The package comes with an evaluator class that supports asynchronous function evaluations. The basic inputs to an asynchronous evaluator class are the *maximum simultaneous* jobs possible on the evaluation platform and the *blocking-fraction* of jobs to be finished before proceeding to the next Bayesian update. For example, the maximum number of jobs on a typical HPC cluster could be 50 but on a local 4 core machine is only 4. The blocking fraction can vary from 0 to 1, with *zero* indicating fully asynchronous update while *one* indicates fully synchronous update. This class has to be derived for a specific platform, examples are provided for a local machine and a SBATCH scheduler based HPC system. On a local machine, methods are implemented to spawn processes for each cost function evaluation. Through the process id, the operating system’s process table is monitored to detect completion of each function evaluation. For a cluster, sample evaluator subclasses are included for an SBATCH based queuing system, for which methods are implemented to track the status of the job through its *jobId*. It comes with a complete setup for performing file transfer and remote job submission, using the Paramiko SSH library for Python. The user only needs to provide external methods that can parse the output files from a simulation and classify them into either of the three categories of: (a) *ValueNotReady*; (b) *EvaluationFailed*; and (c) *EvaluateAgain*;

Jobs with *ValueNotReady* are pushed into the pending list for future Bayesian assimilation. If the evaluation is completed, the result parsing script (user-provided) is invoked and it either returns a cost-function value (float) or either of *EvaluationFailed* or *EvaluateAgain*. Jobs with an *EvaluationFailed* status will not be evaluated in the future. This is done by assimilating the failed evaluation points into the surrogate using a *Kriging Believer* [5] strategy, i.e., we place full belief in the

²This also prevents an iteration from completing with mostly stale points.

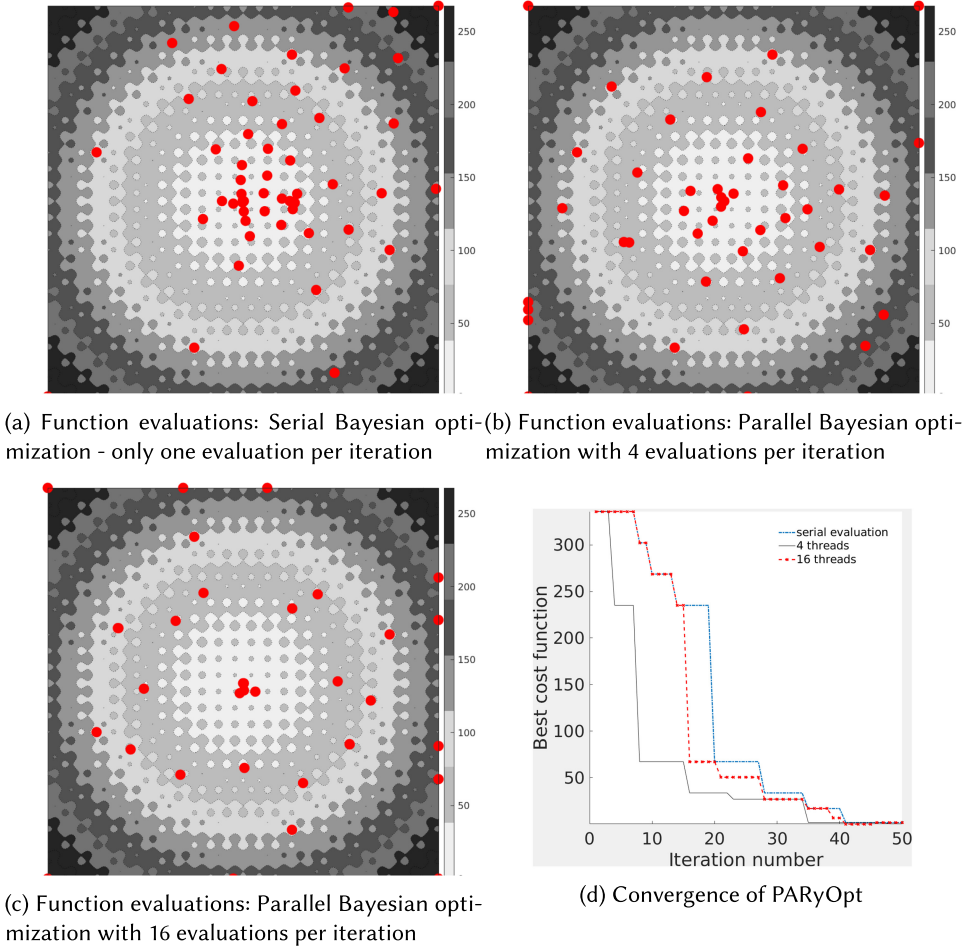


Fig. 3. Bayesian optimization on Rastrigin function with different levels of parallelization. Note that within the given budget, both serial and parallel implementations attain the optimum without any difficulty. While the nature of the convergence is dependant on the cost function and initial guess, it is important to note that PARyOpt attains the optimum in all parallelization scenarios.

existing kriging surface to accurately represent the value at the failed evaluation point. This strategy naturally avoids future evaluation at the failed points. Those with an *EvaluateAgain* status will be submitted again.

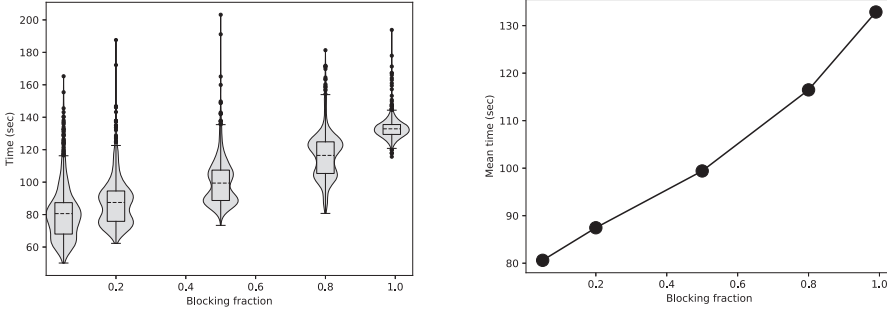
4 RESULTS AND DISCUSSION

All the experiments were performed on the Iowa State computation facility. These are Intel Skylake 6140 processors, with 36 processors per node, 192GB memory per node and 100G IB interconnect.

4.1 Optimization

Finite Budget Optimization. We first show that parallelization does not affect the ability of the algorithm to find true global optima. Optimization was performed on the standard Rastrigin cost function over a domain $(x, y) \in [-12, 12] : \times [-12, 12]$.

$$f(x, y) = 20 + x^2 + y^2 - 10\cos(2\pi x) - 10\cos(2\pi y)$$



(a) Distribution of total optimization time for Bayesian optimization with asynchronous function evaluations

(b) Mean of completion times of Bayesian optimization with asynchronous function evaluations

Fig. 4. Asynchronous Bayesian optimization: times of completion for fixed budget optimization.

The global minimum is located at $(0, 0)$ with a function value of 0.0. No asynchronicity was used and all the evaluations were used to update the surrogate. In all the cases, a *squared exponential* kernel function was used with *LCB* as the acquisition function. Across all runs, the same κ strategy was used for uniformity. Two different levels of parallelization were used: the first with 4 in-fill points per iteration and the second with 8 in-fill points per iteration. Both these cases were compared with a standard serial Bayesian optimization. Figure 3 plots the locations of function evaluations with different levels of parallelization. It should be observed that by using a parallel optimizer, the optima is not compromised. The path of evaluation is indeed affected by the degree of parallelization. The simultaneous identification of in-fill points helps to mimic exploration-exploitation search without explicitly tailoring the κ -strategy.

Asynchronous Evaluator. Next, we demonstrate the performance and advantages of our asynchronous evaluator. Using the same Rastrigin function, we show how asynchronous optimization can improve total optimization times under uncertain function evaluation time. Since the cost function in this example is cheap, function evaluations are instantaneous. Hence, to demonstrate asynchronous working, we artificially enforce a finite total evaluation time. For this, we sample times from a normal distribution with a mean of 10 sec and a standard deviation of 2.5 sec. This distribution is representative of the queue wait time in standard schedulers. Other times, such as the time of evaluation and time of startup are all assumed constant. The mean time of 10 sec is chosen so that the total time is small and simultaneously not too small to be overshadowed by program (Python, here) start up time. As with the previous test, a *squared exponential* kernel function was used along with a *LCB* acquisition function. Since the completion time (wait time, here) of each evaluation is (deterministically) uncertain, several (200) realizations of the optimization were performed to get trends in total optimization time. These realizations were initialized with Latin Hypercube sampling, resulting in consistent but different initial guesses across the realizations. Figure 4 shows the total time of completion of the optimization for different values of blocking fractions. Barring the few outliers (denoted by dots in Figure 4(a)), it can be observed that there is a consistent reduction of total optimization time with reducing blocking fraction. The mean completion time shows an interesting linear trend with blocking fraction, indicating a strong advantage of asynchronous optimization over a fully blocking optimization. In this simple case, the improvement is up to 50% on average compared to the synchronous implementation. However, while there is a significant reduction of the average time of completion, the uncertainty increases for lower blocking fractions. This is an expected outcome of the current test case, the

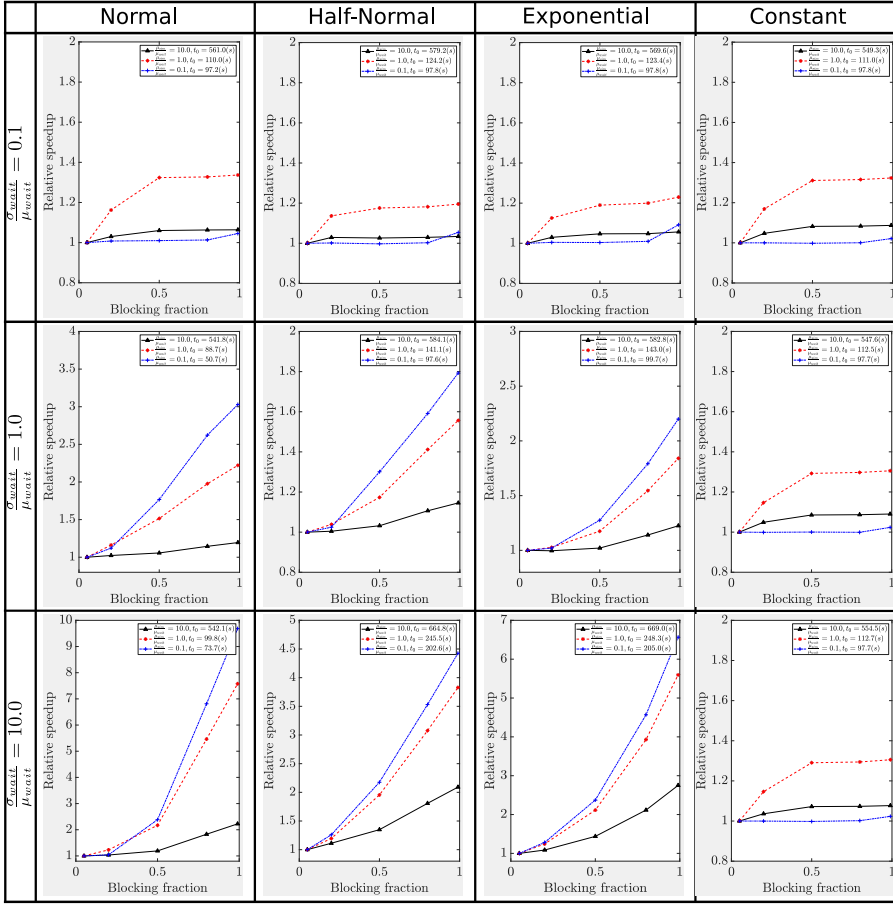


Fig. 5. Performance of the asynchronous evaluator with different wait time distributions and simulation times. Notice that the maximum speedup is obtained for longer and broader wait time distributions (blue curve and bottom row). Please zoom into the legend, for clearer definition of actual normalization times for each panel.

asynchronous evaluator is highly influenced by the wait times more than the fully blocking optimization. Finally, it should be noted that in the worst case scenario, the total completion times for all blocking fractions were similar and not affected by the presence of asynchronicity.

The asynchronous evaluator was tested with more realistic wait and simulation times. We performed a rigorous testing of the evaluator, with different wait time distributions, viz. normal, half-normal [8], exponential [8] and constant, different average simulation time to wait time ratios and different width of distributions. The results are presented in Figure 5. It should be noted that the maximum performance gain is for wait time dominated scenarios with large width of wait time distributions. The effective time improvement is hardly noticeable with narrow wait time distributions.

Benchmark Optimization. Here, we show the working of the software with standard benchmark functions and how the accuracy is affected by the dimensionality of the problem. More specifically, we look at Ackley, Rosenbrock, Rastrigin and Griewank functions, which are known to cause difficulties in convergence with standard optimization methods. Tables 1 and 2 show the results. In

Table 1. Results from Optimization of Benchmark Optimization Test Functions: 2D

Cost function	Optima location	Located Optima
Ackley function	(0.0, 0.0)	(1.35e-11, -2.95e-08)
Rosenbrock function	(1.0, 1.0)	(0.9999999945, 1.00000000124)
Rastrigin function	(0.0, 0.0)	(1.97e-08, 1.17e-09)
Griewank function	(0.0, 0.0)	(-0.0003, -0.0009)

We performed a fixed budget optimization, using 20 serial evaluations.

Table 2. Results from Optimization of Benchmark Optimization Test Functions: Higher Dimensions

Cost function	Optima location	Distance of Located Optima from true optima (l_∞)
Ackley function	$x_i = 0.0$ for $i \in 1, 2, \dots, N$	N=5D 1.75e-9
		N=10D 1.57e-7
		N=20D 1.57e-4
Rosenbrock function	$x_i = 1.0$ for $i \in 1, 2, \dots, N$	N=5D 1.25e-9
		N=10D 3.62e-7
		N=20D 5.7e-4
Rastrigin function	$x_i = 0.0$ for $i \in 1, 2, \dots, N$	N=5D 1.136e-7
		N=10D 4.23e-6
		N=20D 3.8e-3
Griewank function	$x_i = 0.0$ for $i \in 1, 2, \dots, N$	N=5D 1.2e-4
		N=10D 2.4e-3
		N=20D 5.9e-3

We performed a fixed budget optimization, with 10D evaluations in D dimensions.

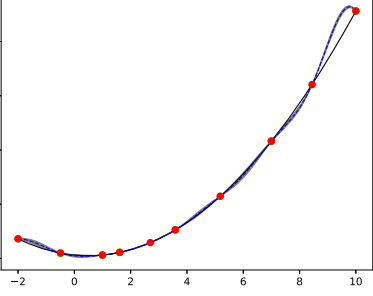
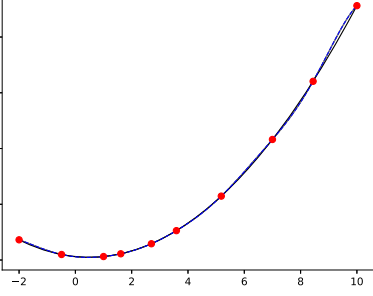
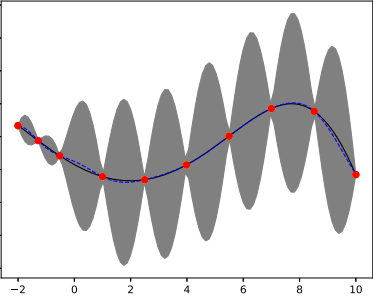
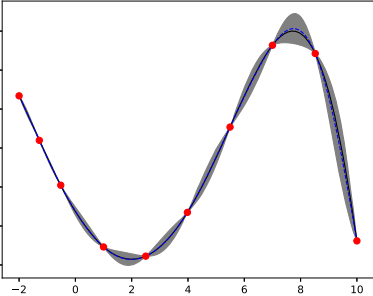
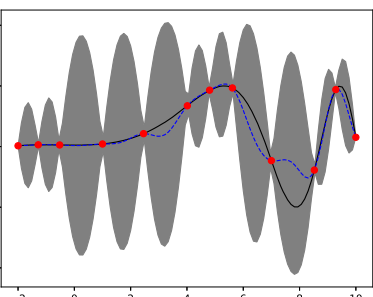
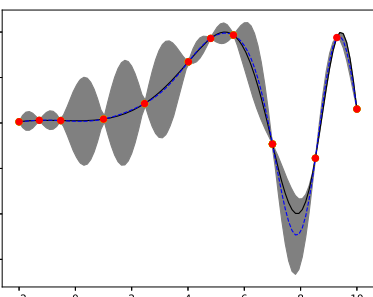
all these cases, no parallelization was used. The *squared exponential* kernel was used with the LCB acquisition function. An annealing type κ parameter [18] was used for the LCB function. It can be observed how with that κ strategy, PARYOpt can simultaneously explore and exploit the search space to efficiently reach the optimum. Also, due to the increased dimensionality of the search space, the accuracy of the located optima suffers – the optimum located by PARYOpt gets farther from the actual minimum when the number of dimensions increase.

4.2 Kriging

Kriging is a more general interpolation technique based on Gaussian priors and posteriors. It informs function values at various locations in terms of mean and variance, and works similar to distance-weighted-interpolation. The algorithmic basis of both Kriging and Bayesian optimization are very similar – use local basis functions to create an interpolant surrogate. In engineering, there are several applications of kriging, especially in the realm of adaptive, automated design of experiments. Due to the well known curse of dimensionality and expensive experiments, engineers look to algorithms that inform regions of maximum new information content. Our current implementation easily enables this by selecting the exploratory mode of acquisition function.

We demonstrate this utility by considering the following example case: what is the most efficient sampling for a given curve (1-D). We shall compare the number of evaluations of Bayesian sampling to uniformly spaced evaluations, typical of response surface reconstruction. We consider purely analytical functions in this study – although their evaluation is cheap, it provides an easy powerful way to see the dependence of number of evaluations on the complexity of the function.

Table 3. Kriging Using PARyOpt

Analytical function	Kriging without MLE optimization	Kriging with MLE optimization
$y = (x - 0.5)^2 + 1$		
$y = \sin(3x^2 + (x - 8)^2 + 1)$		
$y = \sin(((x - 6)/40)^2 + ((2x + 1)/10)^3)$		

In these examples, we perform a budgeted Kriging, i.e., the total number of function evaluations are fixed. Note how hyper-parameter optimization using MLE helps to build greater confidence in the constructed surrogate.

Table 3 shows how the current software can be used for kriging. Each row shows varying modality of the underlying curve. The left column has the result with unoptimized length parameters and the right column shows the result with optimized (kernel) parameters. Note the importance of performing MLE minimization for achieving efficient reconstruction using Kriging. All the results were done with a pure exploration mode using the *LCB* acquisition function.

5 CONCLUSIONS AND FUTURE WORK

In this work, we presented software for performing parallelized asynchronous Bayesian optimization. Several advantages of such asynchronous evaluations are discussed and presented through test examples. Specifically, we showed that asynchronous implementations give a definite speed

up of the total optimization time. This speedup is up to 10x when the wait time distribution of the job queue is very spread out. This framework is completely modular, giving the user the ability to replace any functionality with tailored functions. This framework is also extensible to perform Kriging. Examples were presented to show how the current framework can do this. In this paper, we showed anecdotally the power of this software to be agnostic to the cost function and that asynchronous evaluations speed up the total optimization time. This is practically demonstrated for real world optimization problems involving complex, parallel science simulators. Examples of PARYOpt's usage can be found in material science [10, 14], particle flows [9], and patient-specific cardiac modelling [12]. This software is freely available on [Bitbucket](#) and the documentation is hosted on [ReadTheDocs](#). We anticipate wide usage of this framework by the materials design community.

REFERENCES

- [1] Romain Benassi, Julien Bect, and Emmanuel Vazquez. 2011. *Robust Gaussian Process-Based Global Optimization using a Fully Bayesian Expected Improvement Criterion*. Springer. 176–190 pages.
- [2] E. Brochu, V. M. Cora, and N. De Freitas. 2010. A tutorial on Bayesian optimization of expensive cost functions, with application to active user modeling and hierarchical reinforcement learning. (2010), 49 pages.
- [3] Adam D. Bull. 2011. Convergence rates of efficient global optimization algorithms. *Journal of Machine Learning Research* 12 (Oct. 2011), 2879–2904.
- [4] Carl Edward Rasmussen and Christopher K. I. Williams. 2004. *Gaussian Processes for Machine Learning*. (2004), 69–106. <https://doi.org/10.1142/S0129065704001899> arXiv:026218253X
- [5] David Ginsbourger, Rodolphe Le Riche, and Laurent Carraro. 2010. Kriging is well-suited to parallelize optimization. In *Computational Intelligence in Expensive Optimization Problems*. Springer, 131–162.
- [6] Janis Janusevskis, Rodolphe Le Riche, David Ginsbourger, and Ramunas Girdziusas. 2012. Expected improvements for the asynchronous parallel global optimization of expensive functions: Potentials and challenges. In *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, Vol. 7219, LNCS. Springer, Berlin, 413–418. https://doi.org/10.1007/978-3-642-34413-8_37
- [7] Donald R. Jones, Matthias Schonlau, and William J. Welch. 1998. Efficient global optimization of expensive black-box functions. *Journal of Global Optimization* 13, 4 (1998), 455–492.
- [8] Kirthevasan Kandasamy, Akshay Krishnamurthy, Jeff Schneider, and Barnabas Poczos. 2018. Parallelised Bayesian optimisation via Thompson sampling. In *Proceedings of the Twenty-First International Conference on Artificial Intelligence and Statistics (Proceedings of Machine Learning Research)*, Amos Storkey and Fernando Perez-Cruz (Eds.), Vol. 84. PMLR, Playa Blanca, Lanzarote, Canary Islands, 133–142. <http://proceedings.mlr.press/v84/kandasamy18a.html>.
- [9] Aditya Kommajosula, Daniel Stoecklein, Dino Di Carlo, and Baskar Ganapathysubramanian. 2019. Shape-design for stabilizing micro-particles in inertial microfluidic flows. (2019), 29 pages. <https://arxiv.org/pdf/1902.05935.pdf>.
- [10] Shi Li, Balaji Sesha Sarath Pokuri, Sean M. Ryno, Asare Nkansah, Camron DeÅŽVine, Baskar Ganapathysubramanian, and Chad Risko. 2020. Determination of the free energies of mixing of organic solutions through a combined molecular dynamics and Bayesian statistics approach. *Journal of Chemical Information and Modeling* 60, 3 (2020), 1424–1431.
- [11] Prasanta Chandra Mahalanobis. 1936. On the generalized distance in statistics. In *Proceedings of National Institute of Sciences*, Vol. 2. National Institute of Science of India, 49–55.
- [12] Joshua Mineroff. 2018. *An Optimization and Uncertainty Quantification Framework for Patient-specific Cardiac modeling*. Ph.D. Dissertation. Iowa State University.
- [13] Michael Mongillo. 2011. Choosing basis functions and shape parameters for radial basis function methods. *SIAM Undergraduate Research Online* 4 (2011), 190–209.
- [14] Spencer Pfeifer, Balaji Sesha Sarath Pokuri, Pengfei Du, and Baskar Ganapathysubramanian. 2018. Process optimization for microstructure-dependent properties in thin film organic electronics. *Materials Discovery* 11 (2018), 6–13.
- [15] Balaji Sesha Sarath Pokuri, Joseph Sit, Olga Wodo, Derya Baran, Tayebbeh Ameri, Christoph J. Brabec, Adam J. Moule, and Baskar Ganapathysubramanian. 2017. Nanoscale morphology of doctor bladed versus spin-coated organic photovoltaic films. *Advanced Energy Materials* 7, 22 (2017), 1701269.
- [16] Lukas Polok, Marek Solony, Pavel Smrz, Viorela Ila, and Pavel Zemcik. 2013. Incremental Cholesky factorization for least squares problems in robotics. *IFAC Proc. Vol.* 8 (2013), 172–178. <https://doi.org/10.3182/20130626-3-AU-2035.00027>

- [17] Rommel G. Regis and Christine A. Shoemaker. 2007. Parallel radial basis function methods for the global optimization of expensive functions. *European Journal of Operational Research* 182, 2 (Oct. 2007), 514–535. <https://doi.org/10.1016/j.ejor.2006.08.040>
- [18] Michael J. Sasena, Panos Papalambros, and Pierre Goovaerts. 2002. Exploration of metamodeling sampling criteria for constrained global optimization. *Engineering Optimization* 34, 3 (2002), 263–278.
- [19] Bobak Shahriari, Kevin Swersky, Ziyu Wang, Ryan P. Adams, and Nando De Freitas. 2016. Taking the human out of the loop: A review of Bayesian optimization. *Proc. IEEE* 104, 1 (2016), 148–175.
- [20] Jialei Wang, Scott C. Clark, Eric Liu, and Peter I. Frazier. 2016. Parallel Bayesian Global Optimization of Expensive Functions. (2016). arXiv:1602.05149 <http://arxiv.org/abs/1602.05149>.
- [21] Ziyu Wang and Nando de Freitas. 2014. Theoretical Analysis of Bayesian Optimisation with Unknown Gaussian Process Hyper-Parameters. (2014). <http://arxiv.org/abs/1406.7758>.
- [22] Holger Wendland. 2005. Scattered data approximation, volume 17 of Cambridge monographs on applied and computational mathematics. *Cambridge University Press, Cambridge* 28 (2005), 29.
- [23] Olga Wodo and Baskar Ganapathysubramanian. 2012. Modeling morphology evolution during solvent-based fabrication of organic solar cells. *Computational Materials Science* 55 (2012), 113–126.
- [24] Kui Zhao, Olga Wodo, Dingding Ren, Hadayat Ullah Khan, Muhammad Rizwan Niazi, Hanlin Hu, Maged Abdelsamie, Ruipeng Li, Er Qiang Li, Liyang Yu, and others. 2016. Vertical phase separation in small molecule: Polymer blend organic thin film transistors can be dynamically controlled. *Advanced Functional Materials* 26, 11 (2016), 1737–1746.

Received September 2018; revised July 2021; accepted November 2021